

NÚCLEO DE PESQUISA: PROMPT ENGINEERING APLICADO À ENGENHARIA DE SOFTWARE E À UTILIZAÇÃO DE IA GENERATIVA

Esta pesquisa acadêmica consolida o estado da arte sobre a aplicação sistemática de Prompt Engineering no contexto do desenvolvimento de software assistido por IA, fornecendo embasamento para a metodologia do Trabalho de Graduação.

A. DELIMITAÇÃO

O Prompt Engineering, no contexto desta pesquisa, é delimitado não como uma interação casual em linguagem natural, mas como uma disciplina formal da Engenharia de Software (Prompts-as-Code). A investigação foca em como a estruturação empírica de entradas (definição de objetivos, contexto sistêmico, restrições sintáticas, exemplos few-shot e decomposição de tarefas) atua como mecanismo de controle para garantir a consistência, reprodutibilidade e segurança dos artefatos de software gerados por Modelos de Linguagem de Larga Escala (LLMs).

B. PERGUNTAS DE PESQUISA

1. Quais arquiteturas estruturais de prompt (ex: Chain-of-Thought, Few-Shot) possuem comprovação empírica para a melhoria na geração, refatoração e teste de código-fonte?
2. Como as práticas de versionamento, avaliação contínua e padronização de prompts podem ser integradas ao fluxo de Engenharia de Software para garantir a reprodutibilidade dos resultados?
3. Como o Prompt Engineering atua como mecanismo de controle de qualidade e quais são os riscos inerentes à subespecificação de prompts no desenvolvimento assistido?

C. SUBTEMAS INVESTIGADOS

- Anatomia do Prompt: Instruções, contexto, restrições, definição de objetivo e especificação de formato de saída.
- Técnicas Avançadas: Few-shot prompting, prompting iterativo, decomposição de tarefas (Chain-of-Thought).

- Aplicações Específicas: Prompts para geração de código, testes automatizados, documentação, análise estática e Code Review.
- Governança: Avaliação de qualidade, consistência das respostas, reprodutibilidade, padronização e versionamento de prompts.
- Riscos: Falhas de segurança por prompts mal especificados e sensibilidade dos modelos a pequenas variações semânticas.

D. ESTRATÉGIA E CRITÉRIOS DE SELEÇÃO

A fundamentação baseia-se em estudos seminais e revisões sistemáticas da literatura (2022-2024) provenientes de repositórios como arXiv, ACM e conferências como NeurIPS e ICSE. A seleção isolou o 'folclore da comunidade' (práticas de fóruns sem medição científica) de evidências empíricas rigorosas que documentam o impacto de frameworks específicos de prompt na precisão do código gerado e na prevenção de alucinações lógicas.

E. FONTES E F. EVIDÊNCIAS EXTRAÍDAS

Fonte 1

Autor(es): WHITE, J. et al.

Ano: 2023

Título: A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT

Periódico: arXiv:2302.11382

Tipo: Estudo Metodológico / Framework.

Evidências: A fonte propõe um catálogo formal de padrões de design para prompts em Engenharia de Software. Comprova que especificar formatos de saída rígidos (Output Automater Pattern) e estabelecer uma 'persona' arquitetural reduz substancialmente respostas inconsistentes. (Evidência principal nas Seções 3 e 4 do documento).

Fonte 2

Autor(es): WEI, J. et al.

Ano: 2022

Título: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Periódico: Advances in Neural Information Processing Systems (NeurIPS)

Tipo: Estudo Empírico Fundacional.

Evidências: Demonstra empiricamente que induzir o modelo a decompor uma tarefa complexa passo a passo (CoT) antes de gerar a resposta final aumenta drasticamente a taxa de sucesso lógico. Na geração de código, isso se traduz em pedir ao modelo para escrever o pseudocódigo ou a lógica do algoritmo em linguagem natural antes de codificar a sintaxe propriamente dita.

Fonte 3

Autor(es): DONG, Q. et al.

Ano: 2023

Título: A Survey on In-context Learning

Periódico: arXiv:2301.00234

Tipo: Revisão Sistemática.

Evidências: Valida que a técnica de Few-Shot Prompting (fornecer de 1 a 3 exemplos de entrada/saída no corpo do prompt) atua como um ajuste fino temporário (In-context Learning). Para a geração de testes de software, incluir um exemplo de como o teste deve mockar o banco de dados reduz as alucinações de bibliotecas inexistentes quase a zero.

Fonte 4

Autor(es): HOU, X. et al.

Ano: 2023

Título: Large Language Models for Software Engineering: A Systematic Literature Review

Periódico: ACM TOSEM / arXiv:2308.10620

Tipo: Revisão Sistemática (SLR).

Evidências: Alerta para o fato de que LLMs são hipersensíveis a variações no prompt. Prompts mal especificados levam à geração de código inseguro (vulnerável a injeção SQL, por exemplo). O estudo recomenda que prompts de geração de código sempre carreguem diretrizes explícitas de segurança e restrições de bibliotecas modernas.

G. ANÁLISE CRÍTICA

Ao separar o conhecimento empírico do senso comum, nota-se que práticas como 'ser educado com a IA' não possuem correlação científica com a melhoria da sintaxe do código gerado. Em contraste, técnicas como Chain-of-Thought (WEI et al., 2022) e In-context Learning via Few-Shot (DONG et al., 2023) apresentam eficácia estatística comprovada. Além disso, a literatura aponta que a padronização e o versionamento de prompts ainda são imaturos na indústria. Muitas equipes tratam prompts de forma efêmera, o que destrói a reprodutibilidade: se o prompt que gerou um microserviço crítico não foi versionado no repositório (junto com a temperatura e a versão exata do LLM), a manutenção futura por outro desenvolvedor assistido por IA será incerta, gerando saídas estruturalmente divergentes.

H. COMPOSIÇÃO ACADÊMICA ÚNICA

O Prompt Engineering evoluiu rapidamente de uma técnica heurística de tentativa e erro para uma disciplina fundamental na Engenharia de Software Assistida por Inteligência Artificial. Estruturar um prompt não se resume a emitir um comando em linguagem natural, mas sim a compilar um contexto determinístico para um modelo estocástico. A literatura estabelece que a anatomia de um prompt de alta eficácia deve conter, invariavelmente: uma definição de objetivo clara, o contexto sistêmico (como o código se integra à arquitetura maior), restrições operacionais (quais bibliotecas usar ou evitar) e a especificação rigorosa do formato de saída (WHITE et al., 2023).

Para mitigar o principal risco dos LLMs — a geração de alucinações sintáticas e falhas de lógica de negócios — estudos empíricos consolidaram o uso de técnicas avançadas. O Few-Shot Prompting demonstrou ser essencial para a consistência estilística, utilizando exemplos prévios dentro da janela de contexto para guiar a formatação do código gerado (DONG et al., 2023). Em paralelo, a aplicação de Chain-of-Thought (CoT) altera o comportamento de geração: forçar o modelo a realizar a decomposição das tarefas (explicando a lógica de roteamento antes de gerar o código do *Controller* em uma API) melhora drasticamente a precisão matemática e lógica dos artefatos produzidos (WEI et al., 2022).

Ao longo do ciclo de vida do software, os prompts assumem funções especializadas. Prompts para geração de testes automatizados, por exemplo, exigem que se passe o código-fonte como contexto, junto com diretrizes estritas sobre cobertura de bordas e *mocks* aceitáveis. Já prompts focados em Code Review devem ser calibrados com instruções para buscar complexidade ciclomática e vulnerabilidades de segurança específicas, mitigando o risco de LLMs endossarem códigos maliciosos (HOU et al., 2023).

Contudo, a efetividade contínua dessas práticas depende da governança. A ausência de reprodutibilidade é a maior fragilidade em ambientes de desenvolvimento assistidos. Prompts mal especificados ou efêmeros geram dívida técnica silenciosa e respostas inconsistentes frente a atualizações subjacentes no LLM. Portanto, o Prompt Engineering requer que os prompts sejam tratados como artefatos primários de software: eles devem ser padronizados, versionados em sistemas de controle (Prompts-as-Code) e sujeitos à mesma avaliação e auditoria aplicadas ao código-fonte humano.

I. IMPLICAÇÕES METODOLÓGICAS PARA A TG

A aplicação prática do Trabalho de Graduação na criação do sistema de gestão da grade horária da FATEC Barueri exigirá que o Prompt Engineering seja internalizado não como um auxílio informal, mas como o alicerce metodológico do experimento.

1. Catálogo e Versionamento: O grupo deverá criar um repositório (ou diretório específico no GitHub) denominado 'Prompt Library'. Cada prompt crítico utilizado para gerar classes Java no Spring Boot, construir *queries* no MySQL ou elaborar as rotinas de front-end deverá ser versionado. Isso garantirá a reprodutibilidade acadêmica do TG.

2. Estruturação Padronizada: Os prompts não poderão ser genéricos. Deverão seguir uma taxonomia rigorosa. Exemplo prático: ao invés de 'Faça um serviço de alocação de salas', o prompt deverá conter: (A) Contexto ('Atuando como Arquiteto Java Spring Boot 3...'); (B) Objetivo ('Gerar um @Service para validação de horário acadêmico'); (C) Restrições ('Utilize Java 21, evite NullPointerException usando Optional'); e (D) Formato de Saída ('Retorne apenas o código Java e os testes unitários via JUnit 5').

3. Uso de CoT e Few-Shot: Para regras complexas da FATEC (como matriz curricular e restrições de carga horária dos professores), a equipe aplicará testes baseados em exemplos prévios (Few-Shot) e pedirá ao Gemini para explicar o algoritmo antes de programar as entidades (Chain-of-Thought), validando a lógica institucional antes da geração da sintaxe.

J. REFERÊNCIAS ABNT COMPLETAS

DONG, Q. et al. A Survey on In-context Learning. arXiv, [S. l.], 2023. Identificador: arXiv:2301.00234. Disponível em: <https://arxiv.org/abs/2301.00234>. Acesso em: 6 out. 2026.

HOU, X. et al. Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology (TOSEM), Nova York, v. preprint, 2023. Disponível em: <https://arxiv.org/abs/2308.10620>. Acesso em: 6 out. 2026.

WEI, J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Em: Advances in Neural Information Processing Systems (NeurIPS), [S. l.], v. 35, p. 24824-24837, 2022.

WHITE, J. et al. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. arXiv, [S. l.], 2023. Identificador: arXiv:2302.11382. Disponível em: <https://arxiv.org/abs/2302.11382>. Acesso em: 6 out. 2026.

K. LACUNAS DE INVESTIGAÇÃO

- A literatura carece de métricas absolutas e padronizadas para avaliar quantitativamente a 'qualidade semântica' de um prompt antes de sua execução (predictive prompt evaluation).
- Ferramentas de versionamento nativo voltadas especificamente para gerenciar o ciclo de vida de prompts (como um 'Git para Prompts' integrado a testes unitários automatizados) ainda estão em estágios incipientes na academia e na indústria.